

AI Financial Dream & Goal Planner — Final Project Report

1. Problem

The system helps fresher users estimate the affordability of Marriage, Car and Home goals using city costs, user-entered monthly salary, saving percentage and goal timelines.

2. Machine Learning

The salary dataset is preprocessed with one-hot encoding for City, Education and Job Role. Three regression models are compared: Linear Regression, Decision Tree Regressor and Gradient Boosting Regressor. Random Forest is intentionally excluded. Models are evaluated using MAE and R^2 ; the selected model minimizes MAE with R^2 as the tie-breaker and is saved as `models/salary_model.joblib`.

3. Salary rule

When the user enters a monthly salary, that exact value is authoritative for the financial plan. ML salary prediction remains a separate reference/tool result and never replaces the entered salary.

4. Financial calculations

Future Cost = Current Cost $\times (1 + 0.06)^{\text{Years}}$. Monthly investment uses the future-value annuity formula with a 10% expected annual return assumption. Available capacity = salary $\times$ saving percentage / 100. The total requirement is the sum of Marriage, Car and Home requirements.

5. Inflation

The project uses a fixed 6% annual inflation rate, represented as `INFLATION_RATE = 0.06`.

6. Feasibility

Achievable: required $\leq$ available. Challenging: required/available ≤ 1.5 . Highly Challenging: otherwise. The UI shows surplus/shortfall and gap-reduction suggestions.

7. Investment categories

Up to 3 years: short-term/lower-volatility/capital-preservation-oriented. 4–7 years: medium-term/diversified balanced. More than 7 years: long-term/diversified growth-oriented. These are educational categories only.

8. Agent and tools

The Agent extracts natural-language inputs, selects tools, and delegates numeric work to deterministic Python functions. Optional Ollama support is local; no paid API is required.

9. RAG

Three local documents are stored under `rag/documents`. The retriever chunks local text and searches a local vector representation. If relevant content is not found, it returns an unavailable response instead of inventing information.

10. API and UI

FastAPI exposes `/plan`, `/salary/predict`, `/agent/plan`, `/rag/query`, `/cities` and `/health`. The HTML/CSS/JavaScript frontend is served by FastAPI.

11. Testing

27 automated tests cover inflation, invalid timelines, savings boundaries, salary-primary behavior, feasibility, model comparison, RAG grounding, unknown RAG questions, prompt injection, recommendation rules and Agent tool selection.

12. Deliverables

Source code, salary dataset, city/goal dataset, saved trained model, preprocessing/training code, API/UI, RAG documents/index, Agent/tools, README, architecture/DFD, test evidence and this report are included in the final project archive. Demo video and live viva remain student presentation deliverables.

13. Limitation

This is an educational simulation, not professional financial advice. Returns are not guaranteed and project cost/return assumptions are not forecasts.